

SYSTEM ARCHITECTURE SPECIFICATION

World Cup Intelligence Platform

Components, data pipeline, state model, and scaling

Author Tobi Smith-Kayode

Role Engineering Lead / Architect

Version 1.0

Status Current

Date June 2026

Portfolio artifact — based on a real, production-grade system designed and built by the author.

Contents

- 1. Architecture at a Glance
- 2. Component Inventory
- 3. Request Lifecycle
- 4. Data Pipeline
- 5. State & Caching Model
- 6. Analytics Subsystem
- 7. AI Subsystem
- 8. Frontend Architecture
- 9. Cross-Cutting Concerns
- 10. Scaling Considerations
- 11. Deployment & Runtime
- Appendix A — Module Map

1. Architecture at a Glance

The platform is a **single-process, server-rendered web application** with no external database. It ingests data from football providers, normalizes it into one in-memory model, computes a forecast, and serves interactive analytics through React Server Components. State that would normally live in a database lives instead as a cached, normalized snapshot plus a derived analytics engine, both pinned to process-global memory.

Property	Choice
Runtime	Next.js 14 (App Router) on Node, single process
Rendering	React Server Components, force-dynamic on analytical routes
Persistence	None at rest — normalized snapshot cached in-process
External deps	Football data providers (HTTP); optional hosted LLM
Statefulness	Warm in-memory cache; cold start re-fetches & re-derives

2. Component Inventory

Component	Responsibility
Adapters	Fetch from each provider and emit a normalized DatasetSnapshot
Store	Hold the active snapshot + per-tournament cache + indexes (globalThis)
Analytics engine	Derive ratings, the match model and the Monte Carlo forecast
Query layer	Shape engine + snapshot data into view models for pages
AI layer	Natural-language Q&A and narrative/insight generation
Pages & API routes	Server-rendered analytical surfaces and JSON endpoints
Instrumentation	Startup hook that loads live data in the background

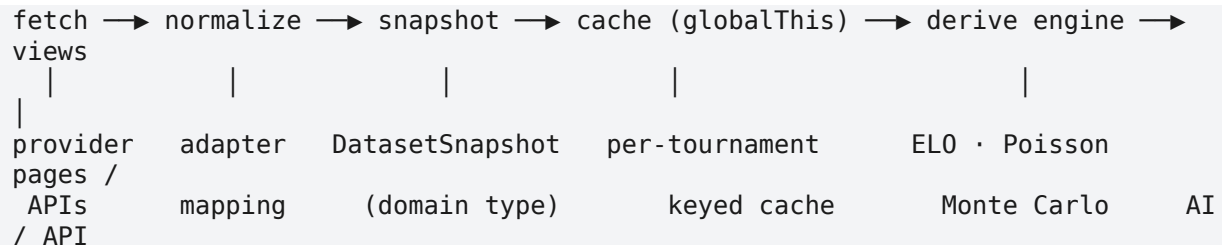
3. Request Lifecycle

A request for an analytical page resolves entirely server-side against warm memory:

1. The route handler (a Server Component) calls the query layer for its view model.
2. The query layer reads the active snapshot from the store and the cached engine output.
3. If the engine is cold, it is computed once and cached on globalThis; subsequent requests reuse it.
4. The component renders to HTML on the server and streams to the client; interactive widgets hydrate.

Why force-dynamic Analytical routes are marked force-dynamic so live data is never frozen into a build-time static page. Static generation would bake whichever dataset existed at build time — fatal for a live-first product.

4. Data Pipeline



- **Fetch** — providers are called by adapters; the live provider is paced and retried.
- **Normalize** — provider payloads become domain entities; missing structure (e.g. groups) is reconstructed.
- **Snapshot** — the unit of truth: a complete, source-agnostic description of one tournament.
- **Cache** — snapshots are stored per tournament; one is "active" at a time.
- **Derive** — the engine computes ratings and the forecast from the active snapshot, lazily and once.
- **Views** — the query and AI layers shape this into what pages render.

5. State & Caching Model

The defining architectural decision is to treat **process-global memory as the system of record** for the active tournament, rather than a database. This is appropriate because the data is provider-owned, refreshes constantly, and is cheap to re-derive.

5.1 Why globalThis

Next.js may evaluate the startup instrumentation hook in a different module instance than request handlers. A value assigned to a module-level variable at boot is therefore not guaranteed to be visible to requests. Pinning the active snapshot, the per-tournament cache and the derived engine to `globalThis` gives every execution context a single, shared source of truth — without standing up Redis or a database for what is effectively a warm cache.

5.2 Multi-tournament keying

- Each tournament's snapshot is cached under its own key; an "active tournament" pointer selects one.
- Switching tournaments swaps the pointer and clears the derived engine so it recomputes for the new dataset.
- All secondary indexes (team-by-id, players-by-team, etc.) are rebuilt atomically when the active dataset changes, preventing stale-index reads.

5.3 Cold vs warm

- **Cold start** — the instrumentation hook begins a non-blocking live load; the server immediately serves a deterministic fallback dataset so it is never unresponsive.
- **Warm** — once the live snapshot lands it becomes active; the engine derives on first request and is cached thereafter.

6. Analytics Subsystem

The engine is a pure function of the active snapshot: ELO ratings, a bivariate-Poisson match model, and an 8,000-run Monte Carlo forecast (championship, stage-reach and group-win probabilities, plus golden-boot projection). It is computed once per dataset and cached alongside the snapshot. Crucially it is **format-agnostic**: group sizes and knockout shape are read from the snapshot, so the same code forecasts a 48-team 2026, a 32-team 2018, or a 24-team women's tournament without branching.

7. AI Subsystem

- **Natural-language query** — an intent parser classifies a question and dispatches to typed handlers that answer directly from the snapshot, returning structured (sentence + table) results.
- **Narrative generation** — a deterministic generator turns engine output into insights, briefings, storylines and match summaries.
- **Optional LLM upgrade** — a thin wrapper can rewrite a pre-computed structured payload into richer prose when a hosted model is configured; the deterministic path is the always-on fallback, so the system never depends on an LLM to function.

Boundary The AI subsystem reads the same normalized data as everything else and never fabricates statistics. This keeps facts verifiable and keeps the LLM strictly optional.

8. Frontend Architecture

- **Server Components by default** — data-heavy pages render on the server against warm memory; the client ships minimal JavaScript.
- **Client islands** — only genuinely interactive surfaces (the tournament switcher, comparison-card builder, the 3D globe, charts) are client components.
- **The 3D globe** — three.js / react-globe.gl, dynamically imported (client-only) so its heavy bundle never blocks server rendering or first paint of other routes.
- **Determinism** — server/client markup is kept hydration-stable (e.g. rounded SVG coordinates, single-template titles) to avoid hydration mismatches.

9. Cross-Cutting Concerns

9.1 Performance

- Warm in-memory reads — no per-request database or network round-trip on the hot path.
- The engine is computed once per dataset and reused across all requests and pages.
- Heavy client bundles (3D) are code-split and loaded only on demand.

9.2 Reliability

- Retry-with-backoff on the live feed; paced batching to respect rate limits.
- Lookup resolution returns "absent" rather than asserting non-null, so transient inconsistency during load degrades gracefully instead of crashing a render.
- Per-surface empty/loading states so a missing entity never fails an entire page.

9.3 Observability

- A health endpoint reports the active competition, team/match counts and live data source.
- Structured server logs surface provider errors and any render-time exceptions for triage.

9.4 Security & ethics

- Provider credentials are server-only environment variables, never shipped to the client.
- The betting surface is analytical only, with non-dismissable responsible-gambling framing and no wager placement.

10. Scaling Considerations

The current single-process design is intentional and sufficient for the product's scale. The path to higher scale is well-defined and does not require re-architecting the core:

Concern	Today	Production path
Horizontal scale	Single process, in-memory cache	Externalize the snapshot/engine cache (e.g. Redis) so instances share warm state
Cold-start cost	Re-fetch + re-derive on boot	Persist the latest snapshot to a store; warm from it on start
Provider limits	Paced, retried in-process	A scheduled ingestion worker decoupled from request serving
Compute	Monte Carlo in request path (cached)	Move forecasting to a background job; serve precomputed results

11. Deployment & Runtime

- **Build** — a production build compiles the app; the live load is skipped at build time so no dataset is baked in.

- **Run** — a single Node process serves the app; the instrumentation hook loads live data in the background on boot.
- **Config** — data source and provider keys are supplied via environment variables; absent a live key, the app runs fully on historical and simulated data.

Appendix A — Module Map

Area	Responsibility
data/providers/*	Source adapters (apiFootball, statsbomb, footballData, generate)
data/store	Active snapshot, per-tournament cache, indexes (globalThis)
analytics/*	ELO, match model, Monte Carlo, derived metrics
server/queries	View models for pages (source-agnostic)
ai/*	Natural-language query + narrative/insight generation
app/*	Server-rendered pages and JSON API routes